

CMSC411: Computer Systems Architecture
Project 3: Cache Coherence
Due: 05/08/2024, 11:59 pm, EST (via Gradescope)

Important policies:

1. Sharing of code between students is viewed as cheating and will receive appropriate action in accordance with University policy.
2. It is acceptable for you to compare your results, and only your results, with other students to help debug your program. It is not acceptable to collaborate either on the code development or on the final experiments.
3. You should do all your work in the C or C++ programming language and should be written according to the C99 or C++11 standards, using only the standard libraries.

Project Description

In this project, you will be creating a simulator that maintains coherent caches for a 4-, 8-, and 16-core processor. The simulator includes the **MI**, **MSI**, **MOSI**, **MESI**, and **MOESI** protocols for a bus-based broadcast (snooping) system.

Specification of Simulator

In this project, we provide you with a simulator that simulates a 4-, 8-, and 16-core processor systems. Each core will consist of a memory trace reader. This trace reader will read in trace files that are also provided to you along with the trace reader.

Each core in the system has one level of cache. The cache is fully associative, has infinite size, and has a single cycle lookup time. The base cache code is provided to you. **You need to implement only the protocol files to process requests at the cache** (described later). The multi-core processor has a single memory controller which can access the off-chip memory. This memory controller is provided for you and will respond to any query (GETS or GETM) placed on the bus with a 100-cycle delay.

The bus is an atomic bus, which indicates that once a query is placed on the bus, the bus will not allow any other requests onto the bus until DATA response is seen. Caches request the bus using a `bus_request()` function. If the bus is not available, it will place the request on an arbitration queue to be scheduled in the future. This is done on a first-come-first-served basis, with node 0 having the highest priority and node N having the lowest priority.

Each processor (i.e., trace reader in the code) will have up to one outstanding memory request at a time. The processor sends a request to the cache and waits until the cache responds with a DATA message. This will be done using the `send_DATA_to_proc()` command.

We have provided a full simulator framework in C++. This framework creates the simulator, reads in the traces, creates a basic cache structure, and creates the memory controller. All of

this code is in the **sim/** directory. **You should not need to change ANY code in the sim/ directory.**

The **protocols/** directory contains files you may need to modify. In other words, you need to complete some of the protocol files (details below).

- When a request comes from a processor to the cache, the cache finds the entry and then calls `process_cache_request()` in the protocol. This is the function that looks at the cache entry's state and decide what messages (if any) should be sent, and what state the cache should go to.
- When a request is snooped on the bus, the cache finds the entry and then calls `process_snoop_request()` in the protocol. It is in this function that you should look at the cache entry's state and decided what messages (if any) should be sent, and what state the cache should go to.

To help you understand the framework, the **MI** and **MSI** protocols are already completed and given to you as an example. You **MUST** fill in the following files:

- **/protocols/MOSI_protocol.cpp**
- **/protocols/MESI_protocol.cpp**
- **/protocols/MOESI_protocol.cpp**

To interface properly with the simulator, do not change any of the class names or delete any functions. The functions you need to fill in (add your code in the numbered sections) fall into two categories:

1. When you are in a state, and you **receive a request**. Depending on the type of the request (LOAD or STORE) you need to decide what to do:
 - `do_cache_I (Mreq *request)` – for MESI, MOSI, and MOESI
 - `do_cache_S (Mreq *request)` – for MESI, MOSI, and MOESI
 - `do_cache_M (Mreq *request)` – for MESI, MOSI, and MOESI
 - `do_cache_O (Mreq *request)` – for MOSI and MOESI
 - `do_cache_E (Mreq *request)` – for MESI, and MOESI
2. When you **snoop a request**. Depending on the type of the request (GETS, GETM, or DATA), you need to decide what to do:
 - `do_snoop_S (Mreq *request)` – for MESI, MOSI, and MOESI
 - `do_snoop_M (Mreq *request)` – for MESI, MOSI, and MOESI
 - `do_snoop_IM (Mreq *request)` – for MESI, MOSI, and MOESI
 - `do_snoop_SM (Mreq *request)` – for MESI, MOSI, and MOESI
 - `do_snoop_E (Mreq *request)` – for MESI, and MOESI
 - `do_snoop_ISE (Mreq *request)` – for MESI, and MOESI
 - `do_snoop_O (Mreq *request)` – for MOSI and MOESI
 - `do_snoop_OM (Mreq *request)` – for MOSI and MOESI

Important notes about simulator assumptions:

- All requests that are not DATA (GETS and GETM) always expect to have someone reply with DATA. To ensure this, the memory will always respond 100 cycles after the request with DATA unless another cache places DATA on the bus first.
There are cases in the traditional protocol where there were certain messages that did not expect replies (e.g. Bus_Upgrade). These types of messages are not supported by the bus and memory, so you cannot use them. Instead you should always send a query that expects a data response (e.g. GETS and GETM). This creates situations where the cache sending the GETS or GETM may be the one that should supply the data. In these cases, the cache should simply send DATA to itself on the bus. Given the difficulties in identifying your own GETS or GETM request, the validations assume that you only reply when you are already responsible for supplying the data.
- In general, there is more than one way to implement each protocol. For this project, the reference implementations were made with simple logic and emphasizing cache to cache transfers, while running in minimal time.

How to run:

In the root directory type `make` and hit enter. This should build an executable in the root directory called `sim_trace`.

Run the simulator using:

```
./sim_trace -t <trace_directory> -p <protocol>
```

As an example, you can test your install by running:

```
./sim_trace -t traces/4proc_validation/ -p MI
```

The **/trace** directory is the directory with the traces you need for running the simulator. The **/trace** directory consists of several sub-directories for validations and experiments for 4-, 8-, and 16-core processors (e.g., `/trace/4proc_validation` and `/trace/experiment4` are for 4-core system). Each of these sub-directories includes one trace for each core in the machine (e.g., `4proc_validation` consists of four “.trace” files while `8proc_validation` includes eight of them) and a config file that simply contains the number of cores for this trace (e.g., 4). Each line in a trace file denotes one memory access and includes the action (read or write) and the address (e.g., `r 0x5e7ddf60`).

The `-p` in command above indicated the protocol you want to run. The options supported by the framework are:

- MI (already implemented)
- MSI (already implemented)
- MOSI (**you need to implement**)
- MESI (**you need to implement**)
- MOESI (**you need to implement**)

Statistics (output)

The simulator outputs the following statistics after completion of the run:

1. Final cache coherence state
2. Number of cycles to complete execution
3. Number of cache misses
4. Number of cache accesses
5. Number of “silent upgrades” for the protocols with E state
6. Number of cache-to-cache transfers (this refers to the number of times data is not supplied by the memory)

Validation

Inside of each **/trace/<n>proc_validation** directory you also find multiple text files (.txt) for the validation runs of each protocol. You should perform experiments on all the provided experiment traces.

Experiments

1. For the 8 experimental traces report the following results when you vary the number of cores. Summarize key results and take-aways in an intuitive manner using appropriate data visualization techniques (e.g., plots and tables) to explain your reasoning.
 - Compare the # of cycles, # cache misses, # cache accesses, and # cache-to-cache transfers for MSI, MESI, MOSI, and MOESI protocols.
 - For MESI and MOESI, report the number of silent upgrades.
2. Based on the results you reported above, come up with some conclusions for various scenarios. For instance,
 - If you were to choose a cache coherence protocol that works well for various traces (i.e., all programs are equally important), which one you choose? -- recall lessons learnt on aggregation and performance evaluation in the early part of the course.
 - What if you want your system to be optimized for certain scenarios?
 - Which protocol you choose for a larger system with more cores?
3. What are the limitations of the simulator? What are some of the enhancements needed to make it more realistic? Answer in a single paragraph.

Deliverables

What to hand in via Gradescope:

- The commented source code for the protocols added to the simulator program. This is the protocols/ folder from your completed implementation:
 - **MOSI_protocol.cpp**
 - **MESI_protocol.cpp**

- **MOESI_protocol.cpp**
- A report (.pdf) that containing the results required by the experiments section above.

Remember that your code must compile and run on a current variant of Linux (i.e., Debian, Red Hat, Ubuntu) running on an x86 architecture (i.e., Intel or AMD).

Grading Rubric

- 0% You do not hand in anything
- +6% Your simulator doesn't run, does not work, but you hand in significant commented code
 - (2%) MOSI: 20 numbered parts in the code
 - (1.8%) MESI: 18 numbered parts in the code
 - (2.4%) MOESI: 24 numbered parts in the code
- +54% Your simulator matches the validation outputs (6% each combination of protocol/#core)
- +40% You completed the experimental section

Important note: Late submissions will be deducted 25% per day